Information Retrieval 1

Document Ranking and Keyword Search

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following book:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Overview of Today's Lecture

We use Google every day to search and instantly find the information we need. The technology behind this "obvious" capability is **Information Retrieval (IR)**.

First Half: Fundamentals of Document Ranking

- What is document ranking?
- Diverse applications of document ranking
- History of information retrieval (1945 to present)
- Limitations of exact matching and the vocabulary mismatch problem

Second Half: How Keyword Search Works

- BM25 scoring function
- Structure and processing of inverted indexes
- Query expansion and RM3
- Search engine implementation infrastructure (Lucene ecosystem)

What Is Document Ranking?

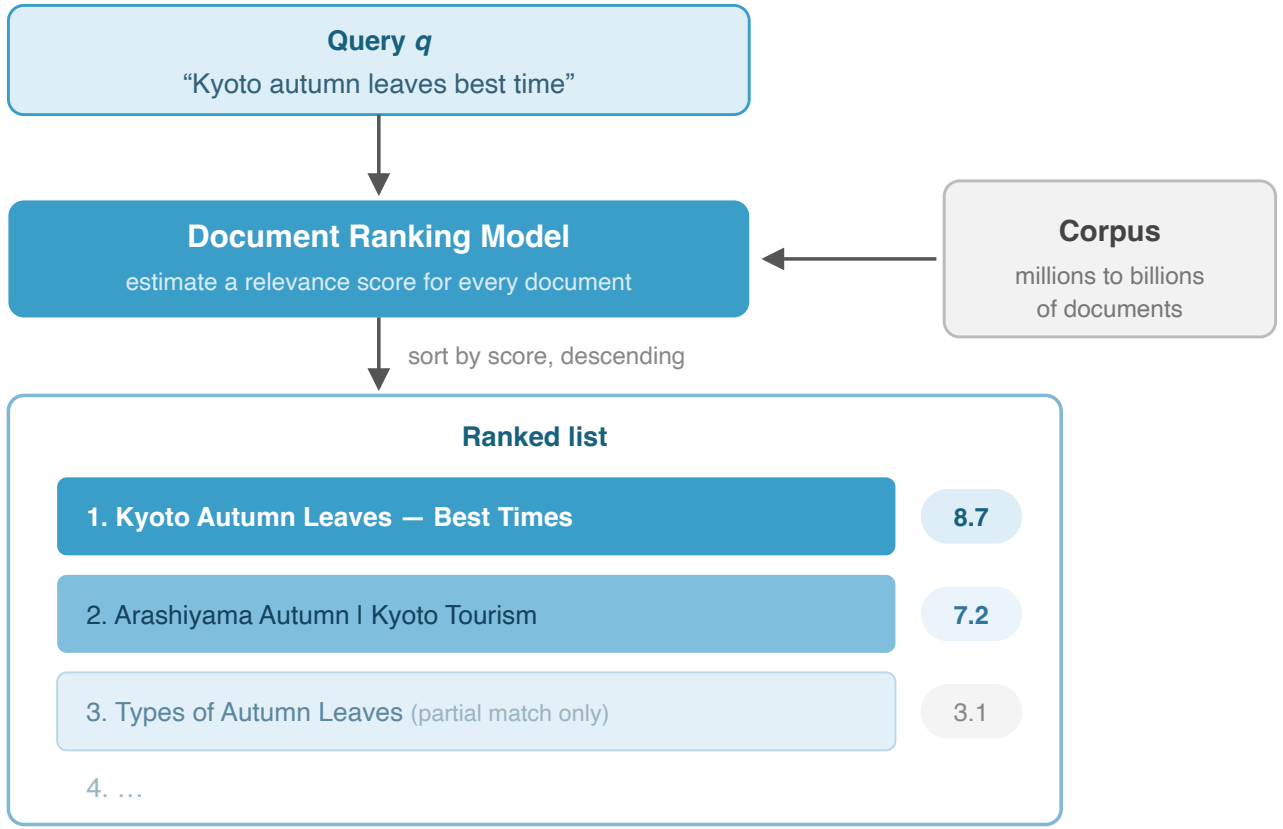
Text Ranking (Document Ranking)
Given a query, produce an ordered list of texts from a corpus, sorted by **estimated relevance**

Most common application: **Search**

Targets: web pages, papers, news, tweets, etc.

Relevant Documents: Satisfy the user's information need

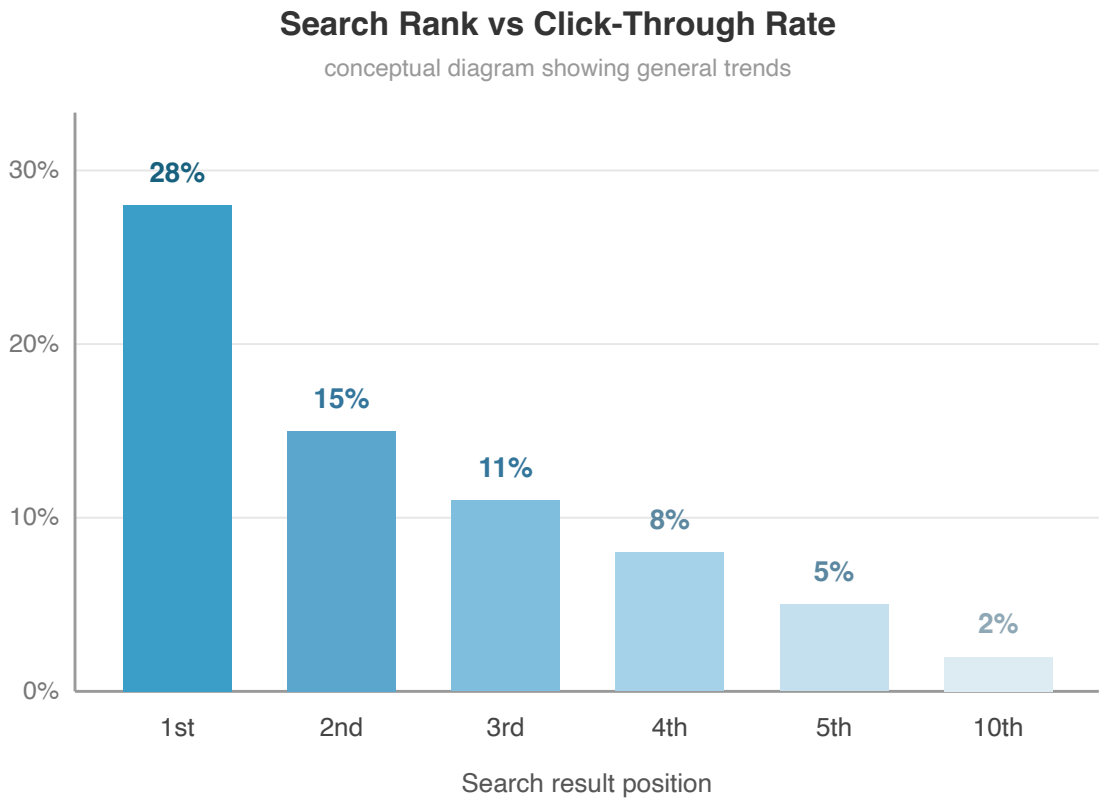
Essence: not filtering, but **ordering**



The goal of ranking is to place highly relevant documents at the top — ordering, not filtering

The order of search results has a decisive impact on user experience

- Users are known to look at only the **top few results**
  - Click-through rates drop sharply as rank position decreases
  - Very few users look beyond the second page
- Simply "finding" relevant documents is not enough
  - **Placing the most relevant documents at the top** is essential
- Ranking quality = Search engine quality
  - The main reason Google won over other search engines was the superiority of its ranking algorithms (PageRank, etc.)



Most users only look at the top 3 results — ranking quality decides search engine quality

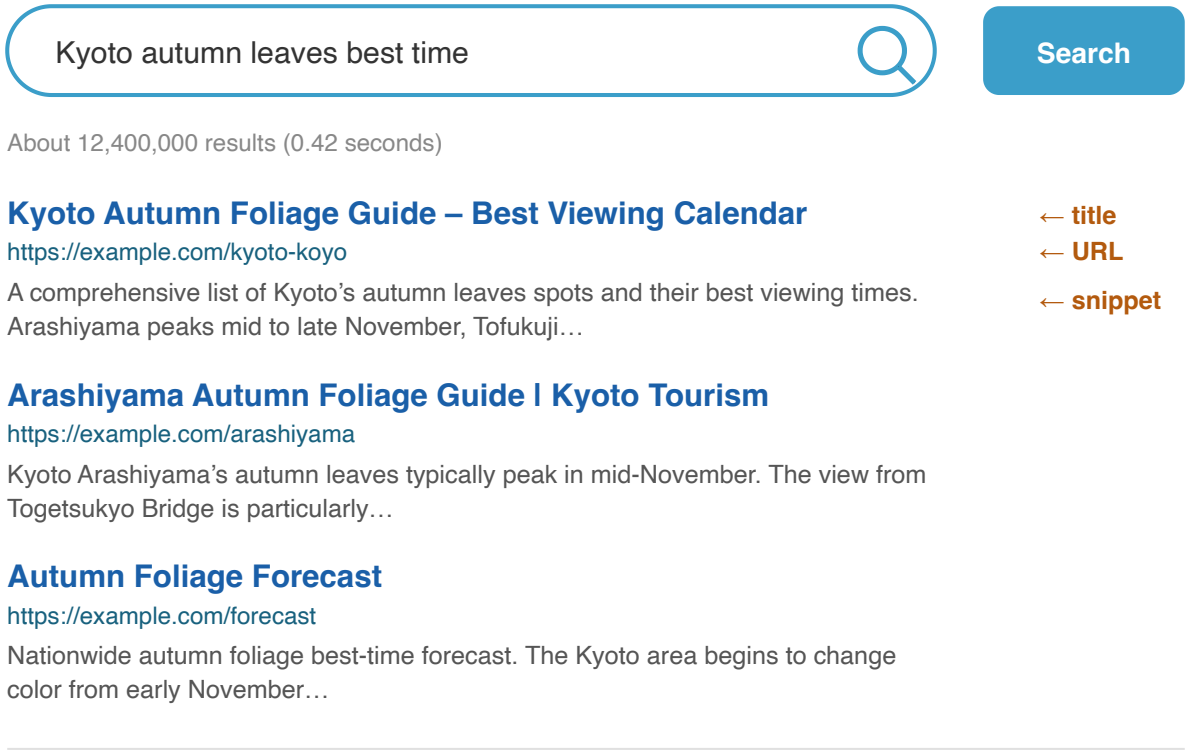
A user enters a few keywords, and results are returned as a ranked list

Input: Keyword query

- User types a few words into the search box
- Example: "Kyoto autumn leaves best time"
- The average web search query is about 2-3 words

Output: Search Engine Results Pages (SERPs)

- Also known as: hit list, "ten blue links"
- Each result consists of a title, snippet, and URL



Input: a few keywords (avg. 2–3 words) → **Output:** SERP — a ranked list of "ten blue links"

Each result consists of a title, snippet, and URL

> **Ad Hoc Retrieval**
> The problem of searching and ranking relevant documents from a static document collection in response to arbitrary queries that are not predefined

The term most frequently used by IR researchers when referring to document ranking

"Ad Hoc" = on-the-spot, not predetermined

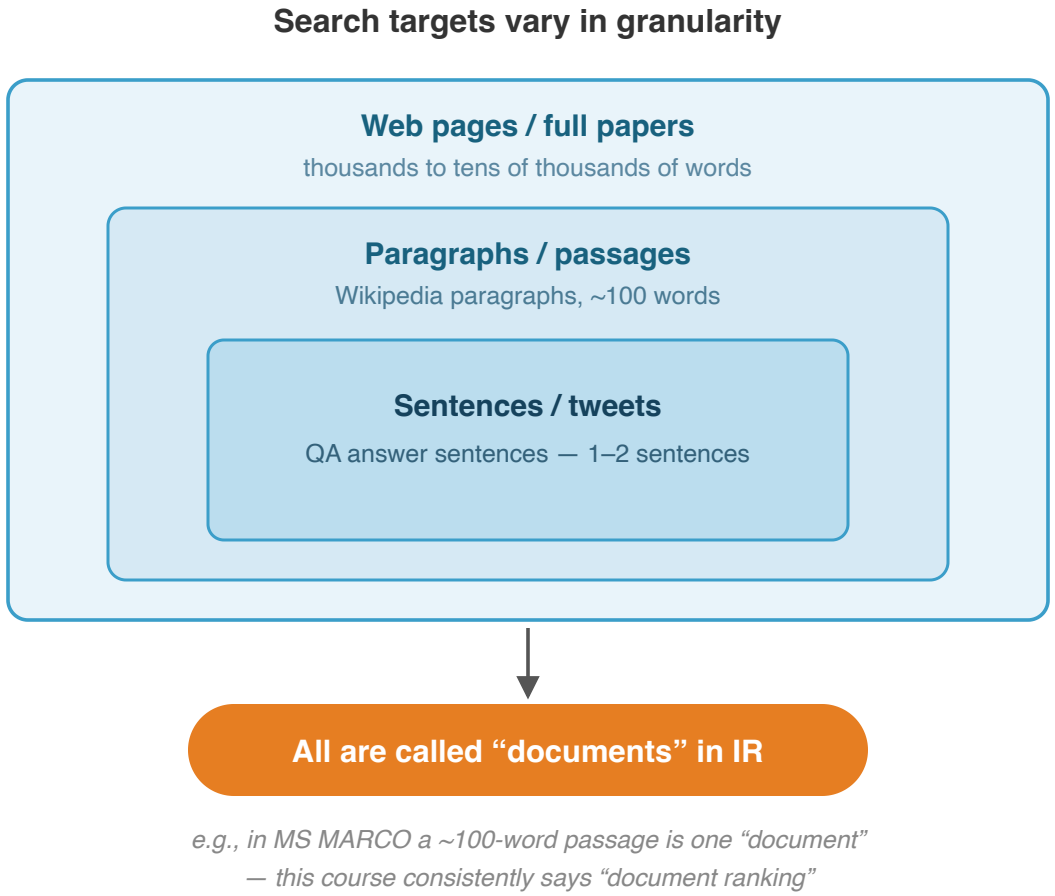
- Queries are not fixed in advance (users input them freely)
- The document collection is built before search time

Google Search, library OPACs, academic paper search (Google Scholar, CiNii) --- all are ad hoc retrieval

Most IR benchmarks (TREC, NTCIR, etc.) assume this problem setting

Search targets vary in granularity, but in IR, by convention, all are called **"documents"**

- The actual search target is not always a full "document"
  - Web pages, full papers (thousands to tens of thousands of words)
  - Passages: a section of a paper, approximately 100 words
  - Sentences / tweets: short text of 1-2 sentences
- However, by IR convention, search targets of any granularity are **all called "documents"**
- This lecture will consistently use the term **"document ranking"**
- Example: In MS MARCO, passages of approximately 100 words constitute a single "document"



Applications of Document Ranking

Document ranking technology is a **foundational technology underlying diverse tasks** beyond search

Applications presented directly to users

- **Question Answering (QA)**: Answers from Siri or Google
- **Community QA (CQA)**: Searching for similar questions on Stack Overflow
- **Information Filtering**: Paper alert notifications
- **Text Recommendation**: Related articles on news sites

Internal system components

- **Input to downstream modules**: Pre-processing for summarization and extraction
- **Semantic similarity comparison**: Determining whether two sentences have the same meaning
- **Fact checking**: Collecting evidence documents
- **Dialogue systems**: Searching for response candidates

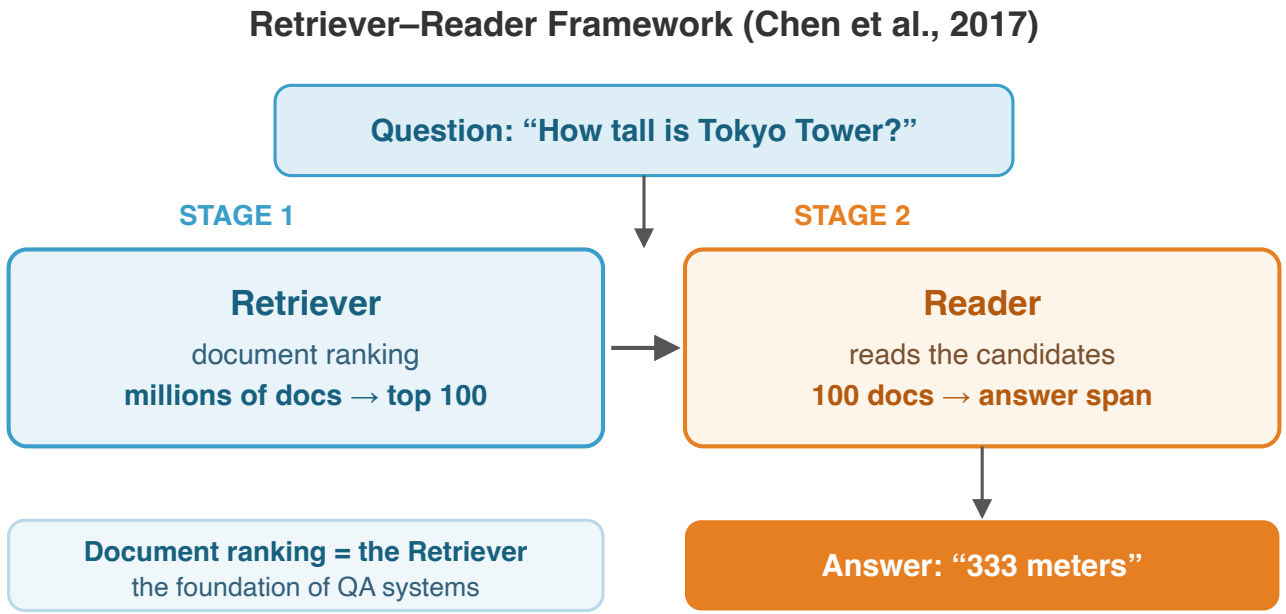
Even when ranked lists are not directly presented to users, they function as critical component technology within larger systems

Question Answering (QA) is one of the most familiar applications of document ranking

- Deployed as Google's **Featured Snippets**
  - Displays direct answers at the top of search results
- Also used for voice assistant read-aloud answers (Siri, Alexa, Google Assistant)

Retriever-Reader Framework [Chen et al., 2017]

- **Stage 1 (Retriever)**: Narrows down candidates from a large document collection via document ranking
- **Stage 2 (Reader)**: Extracts the answer span (specific location) from candidate documents



Document ranking handles the **Retriever (Stage 1)**

Chen et al., "Reading Wikipedia to Answer Open-Domain Questions", ACL, 2017

Community QA

The problem of searching for similar existing questions on Q&A sites such as **Quora**, **Stack Overflow**, and **Yahoo! Answers**

Determining "Has the same question been asked before?"

Estimating **semantic similarity** between texts is at the heart of this task

Example: "How do I sort a list in Python?" and "How to reorder an array in Python" are the same question despite different wording

Semantic Similarity Comparison

Whether two texts "mean the same thing" is a fundamental NLP problem

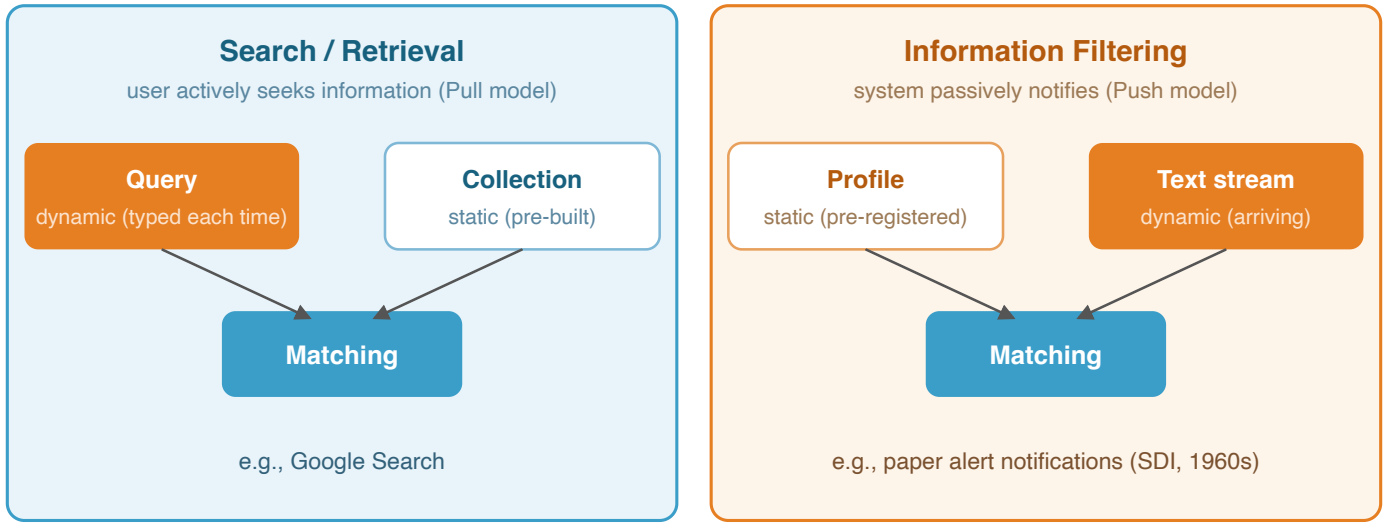
The boundary between document ranking and NLP is becoming blurred

Applications:

- **Paraphrase Detection**
- **Duplicate Question Detection**
- **Natural Language Inference (NLI)**

Search and Filtering Are "Two Sides of the Same Coin"

Search and information filtering have opposite structures but are essentially the same problem



Opposite structures — what is **dynamic** and what is **static** are swapped — but essentially the same matching problem

	Search	Filtering
Query	Dynamic (entered each time)	Static (pre-registered)
Collection	Static (pre-built)	Dynamic (stream)
User action	Active (Pull)	Passive (Push)
Example	Google Search	Paper alerts

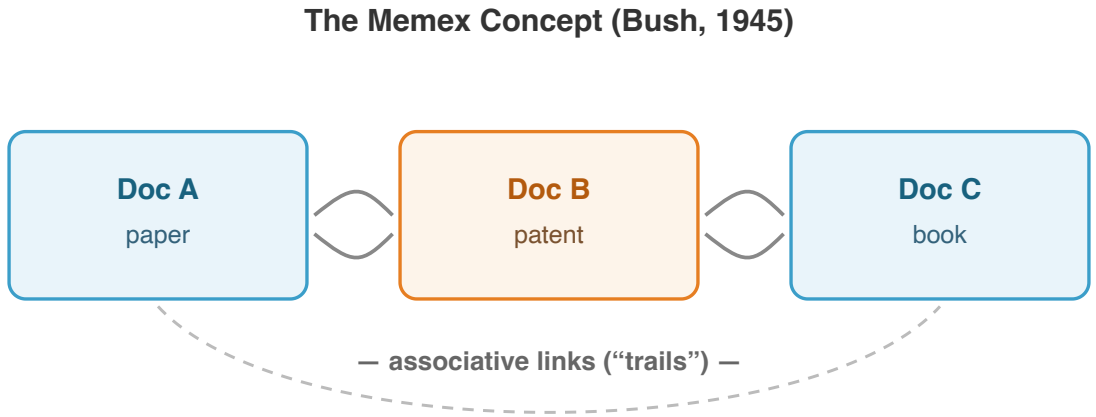
In the 1960s, this was called SDI (Selective Dissemination of Information)

Belkin, "Intelligent Information Retrieval", 1992

History of Information Retrieval

Even before the advent of computers, concern about information explosion gave rise to the vision of information retrieval

- **Vannevar Bush** (MIT Vice President, Director of U.S. Office of Scientific Research)
- 1945: Published **"As We May Think"** in The Atlantic
- **Memex**: A hypothetical machine linking documents via associative indexes — often regarded as the prototype of hypertext
- Advocated that machines should assist humans in recording and referencing knowledge



Users record connections between documents and explore knowledge by following links

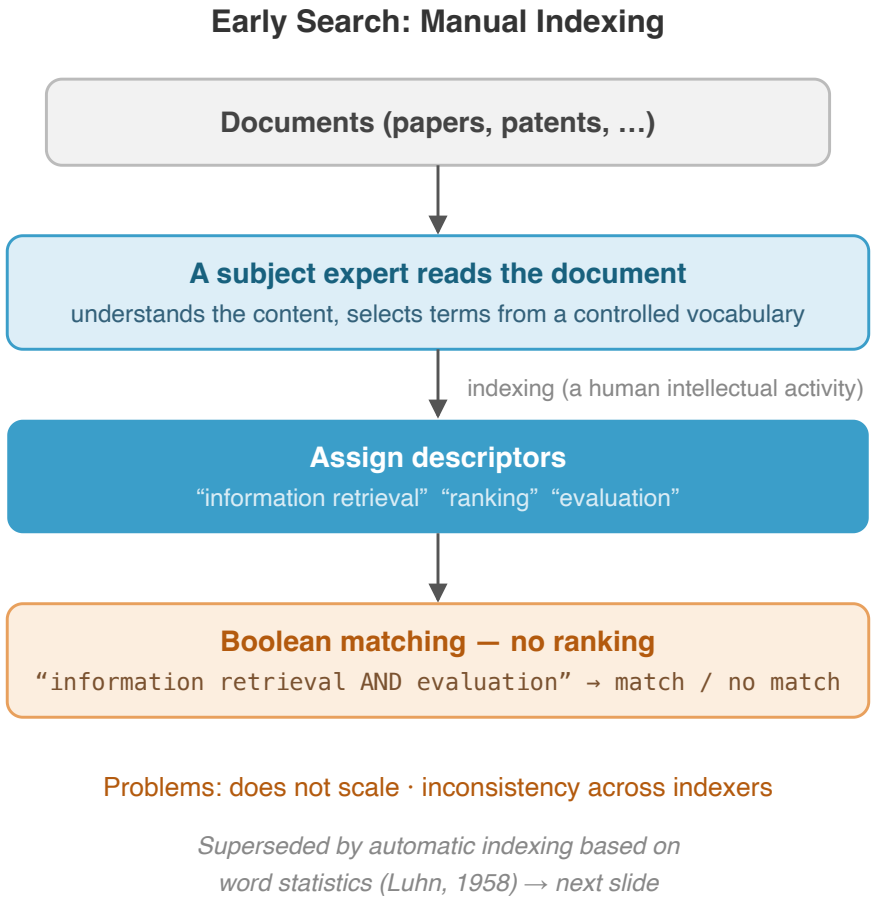
→ **the intellectual origin of hypertext and the WWW**

Bush, “As We May Think”, The Atlantic, 1945 — envisioned before electronic computers existed

Bush, "As We May Think", The Atlantic, 1945

The earliest search systems had human experts assign "descriptors" to documents, then matched against them

- **Descriptors**: Experts selected terms from controlled vocabularies and assigned them to documents
- This process was called **indexing** (a human intellectual activity)
- Search: **Boolean matching** (AND/OR/NOT) — no ranking
- Problems: Does not scale; inconsistency across indexers



In the 1950s-60s, automatic indexing based on statistical analysis of words was proposed

Luhn [1958] --- Pioneer of Statistical Approaches

IBM researcher; proposed extracting important terms automatically based on word **frequency and distribution**

Described ideas that preceded TF-IDF weighting

No implementation or evaluation; it remained conceptual

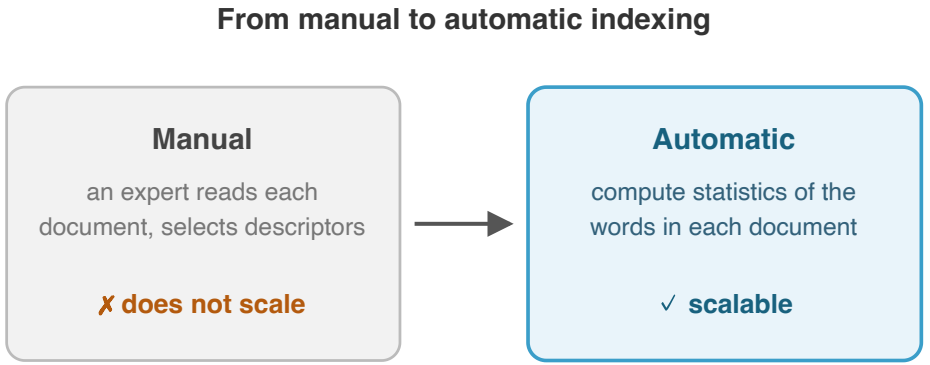
Also proposed automatic document summarization

Maron & Kuhns [1960] --- Quantifying Relevance

Proposed computing a **"relevance number"** from index terms and ordering documents accordingly

The starting point of modern scoring and ranking

A crucial conceptual leap: words automatically extracted from documents can function as descriptors



Words automatically extracted from documents can function as descriptors

Luhn, "The Automatic Creation of Literature Abstracts", IBM J. Res. Dev., 1958
Maron & Kuhns, "On Relevance, Probabilistic Indexing and Information Retrieval", JACM, 1960

In the 1960s-70s, the effectiveness of automatic indexing was experimentally demonstrated

Gerald Salton and SMART

Gerald Salton (Cornell) — the **"Father of IR"**

Developed **SMART**, a vector space model-based automatic retrieval system

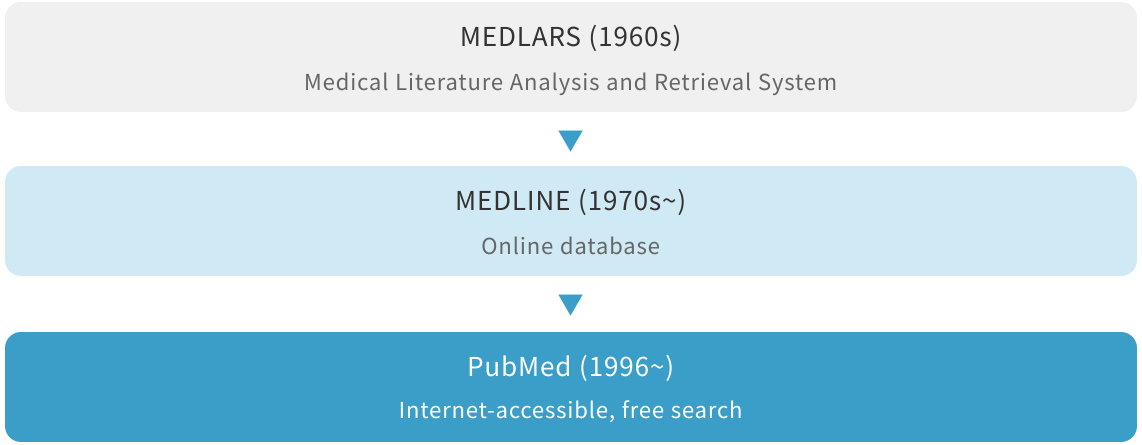
Demonstrated that **automatic indexing performed as well as manual indexing**

The Cranfield Experiments

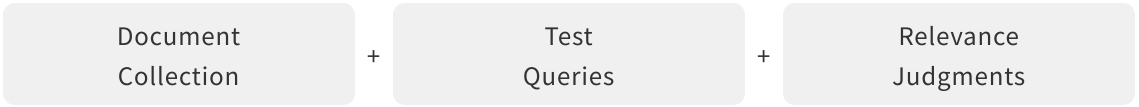
Cyril Cleverdon (Cranfield University, UK)

Cranfield Paradigm: Test collections (docs + queries + relevance judgments) — detailed in Lecture 3

Evolution of MEDLARS



The Cranfield Paradigm



In the 1970s, the foundations of document representation and weighting methods still widely used today were established

Sparck Jones [1972] --- IDF

- **Karen Sparck Jones** (Cambridge): **IDF** — terms in many documents have lower discriminative power; rare terms get higher weights

Salton et al. [1975] --- Vector Space Model

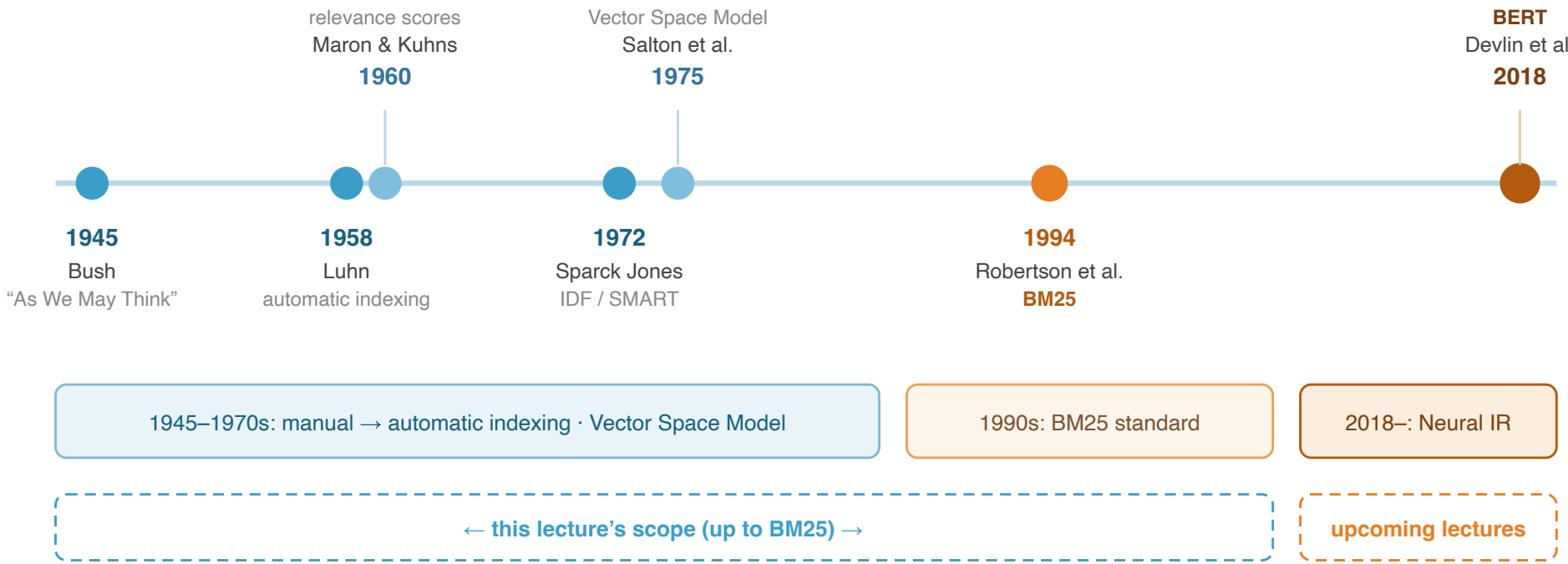
- Documents and queries as **bag-of-words** sparse vectors
- Weights: **TF-IDF** (term frequency x inverse document frequency)
- Similarity: **Cosine similarity** (inner product)

TF-IDF Intuition

Term	TF (term freq.)	DF (doc. freq.)	TF-IDF
Kyoto	High	Low	**High**
autumn leaves	High	Low	**High**
the	High	Very high	Low
is	High	Very high	Low

Assigns high weights to **terms that characterize a document**

Sparck Jones, "A Statistical Interpretation of Term Specificity and Its Application in Retrieval", J. Doc., 1972
Salton et al., "A Vector Space Model for Automatic Indexing", CACM, 1975



1945-1970s: Manual -> automatic indexing, establishment of the vector space model

1990s: Probabilistic models (BM25) become the standard for practical search

2018~: Pretrained Transformers (BERT) dramatically improve ranking accuracy -> **Covered in upcoming lectures**

Limitations of Exact Matching and the Vocabulary Mismatch

A common characteristic of all approaches so far: dependence on **exact term matching**

If a document's terms do not **exactly match** the query's terms, they do not contribute to the relevance score

General form of the scoring function:

$$S(q,d) = \sum_{t \in q \cap d} f(t)$$

$q \cap d$: Terms **shared** between query and document

$f(t)$: Function of term statistics — **TF** (term frequency in doc), **DF** (number of docs containing term), **document length**

Exact Match Scoring

Query: **"Kyoto autumn leaves famous spots"**

Document: "**Kyoto**'s Arashiyama is **famous** for **autumn leaves**"

-> 3 terms match -> Contributes to score ✓

Document: "**Kyoto** temples with beautiful foliage locations"

-> Only 1 term matches -> Low score ✗

"foliage" != "autumn leaves", "locations" != "famous spots"
-> Same meaning but no match

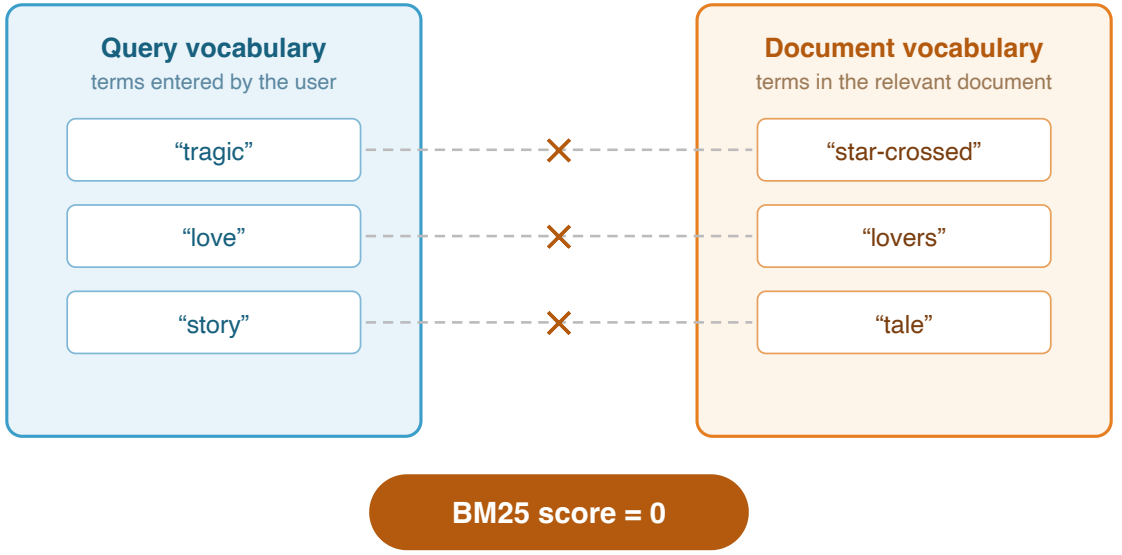
Stemming / **morphological analysis** can partially address this, but they are powerless against paraphrasing

The fundamental limitation of exact match search: searchers and document authors express the **same concept with different words**

Furnas et al. [1987]: People use vastly different words for the same concept

Searcher's words != Document's words → exact matching misses relevant docs

Query	Document (synonym)
moving	relocation, transfer
automobile	car, vehicle
side effects	adverse events



"tragic" ≈ "star-crossed", "love" ≈ "lovers", "story" ≈ "tale"
— semantically close, yet no exact term match

Exact-match search can never find this relevant document (Furnas et al., 1987)

Furnas et al., "The Vocabulary Problem in Human-System Communication", CACM, 1987

Three main research directions have been pursued to address the vocabulary mismatch problem

1. Enriching Query Representation

Query expansion: Adding related terms to the original query

Relevance feedback: Extracting terms from relevant documents [Rocchio, 1971]

Pseudo-relevance feedback: Expanding using top-ranked documents assumed to be relevant [Croft & Harper, 1979]

-> Covered as RM3 in the second half of this lecture

2. Enriching Document Representation

Enrich document representation by adding related terms and metadata

Particularly effective for short documents and speech transcripts

Use of bibliographic metadata [Kwok, 1975]

Revived as **document expansion** in the Transformer era

3. Going Beyond Exact Matching

Latent Semantic Analysis (LSA) [Deerwester et al., 1990]

Topic Models (LDA) [Wei & Croft, 2006]

Statistical Translation Models [Berger & Lafferty, 1999]

-> Evolved into Neural IR (Dense Retrieval)

Approaches 1 and 3 are currently mainstream. In particular, Neural IR aims to fundamentally solve the vocabulary mismatch -> **Covered in upcoming lectures**

BM25

$$\mathrm{BM25}(q,d) = \sum_{t \in q \cap d} \underbrace{\log \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}}_{\text{IDF component}} \cdot \underbrace{\frac{\mathrm{tf}(t,d) \cdot (k_1 + 1)}{\mathrm{tf}(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{l_d}{L}\right)}}_{\text{TF saturation + document length normalization}}$$

BM25 = Best Match 25: Named after the Okapi family of probabilistic ranking functions (the literature often describes it as the "25th" variant)

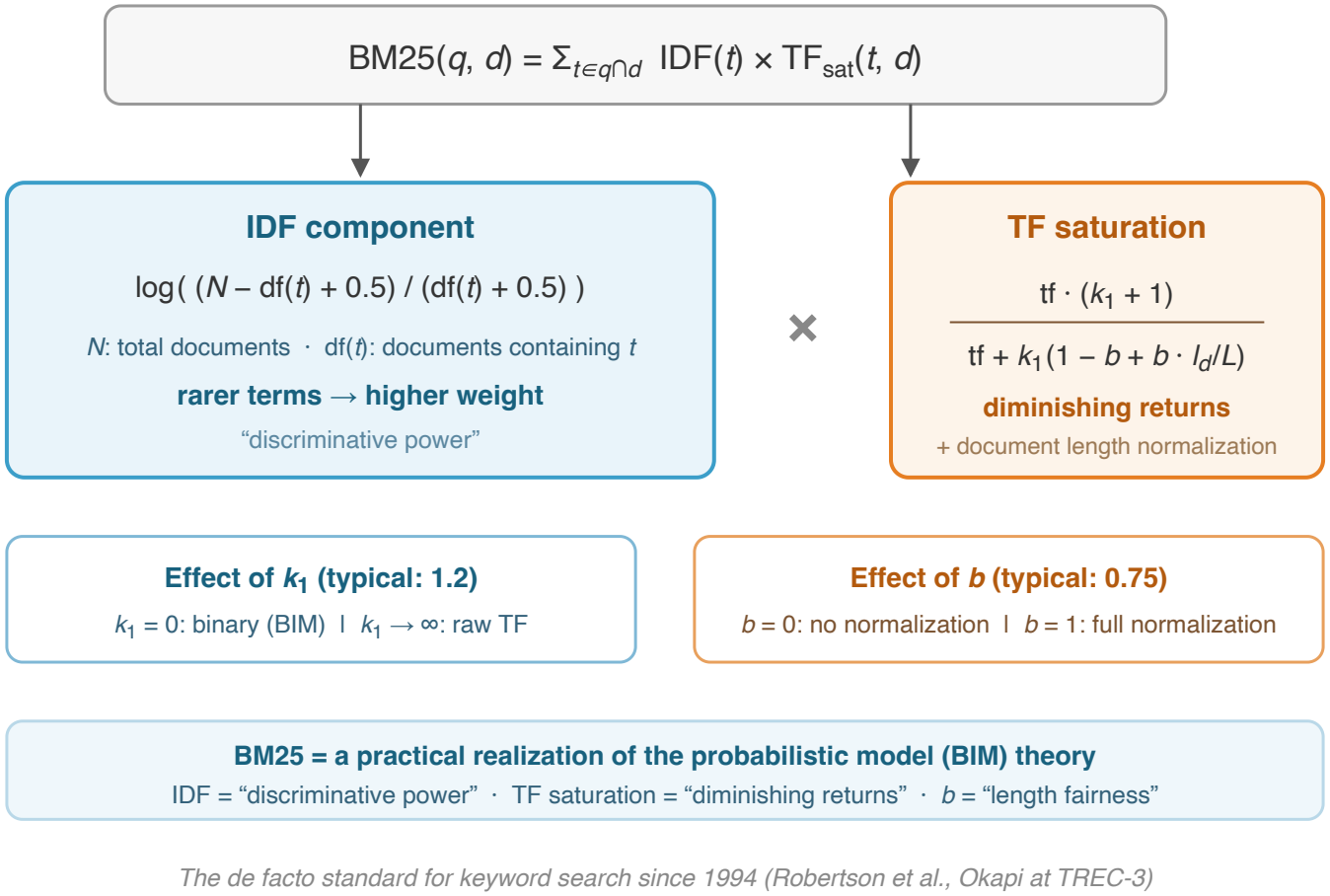
Structure: sum of **IDF component x TF component** for each query term

Since its proposal in 1994, it has been the **de facto standard** for keyword search

Implemented as standard in major search engines: Elasticsearch, Solr, Lucene, etc.

Robertson et al., "Okapi at TREC-3", TREC, 1994

BM25 Scoring Structure



The de facto standard for keyword search since 1994 (Robertson et al., Okapi at TREC-3)

Product of two components:

1. **IDF**: Discriminative power of a term. Rarer terms get higher weights
2. **TF saturation**: Diminishing effect of term frequency + document length normalization

Parameters:

$k_1 = 1.2$: Speed of TF saturation

$b = 0.75$: Degree of document length normalization

BM25 is derived from probabilistic model theory (BIM) (-> detailed in Lecture 4)

Robertson and Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond", FnTIR, 2009

Rarer terms are more important for determining document relevance

$$\mathrm{IDF}(t) = \log \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}$$

N: Total number of documents in the corpus

$\mathrm{df}(t)$: Number of documents containing term t

Small $\mathrm{df}(t)$ (rare term) -> Large IDF

Large $\mathrm{df}(t)$ (common term) -> Small IDF

Intuition: Common words ("the", "is") are useless as search clues; specific terms ("Kyoto") appearing in few documents are informative

IDF Values (e.g., N = 100,000)

Term	df(t)	IDF	Interpretation
Kyoto	5,000	1.28	**High** (discriminative)
tourism	10,000	0.95	Medium
Japan	30,000	0.37	Low
the	99,000	-2.00	Negative (appears in almost all docs)

When $\mathrm{df}(t) > N/2$, IDF becomes negative -> typically removed as stop words

log is the common logarithm (base 10), consistent with the exercises

The score increases as term count grows, but the increase diminishes (saturation)

$$\text{TF component} = \frac{\text{tf}(t,d) \cdot (k_1 + 1)}{\text{tf}(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{l_d}{L}\right)}$$

$\text{tf}(t,d)$: Number of occurrences of term t in document d

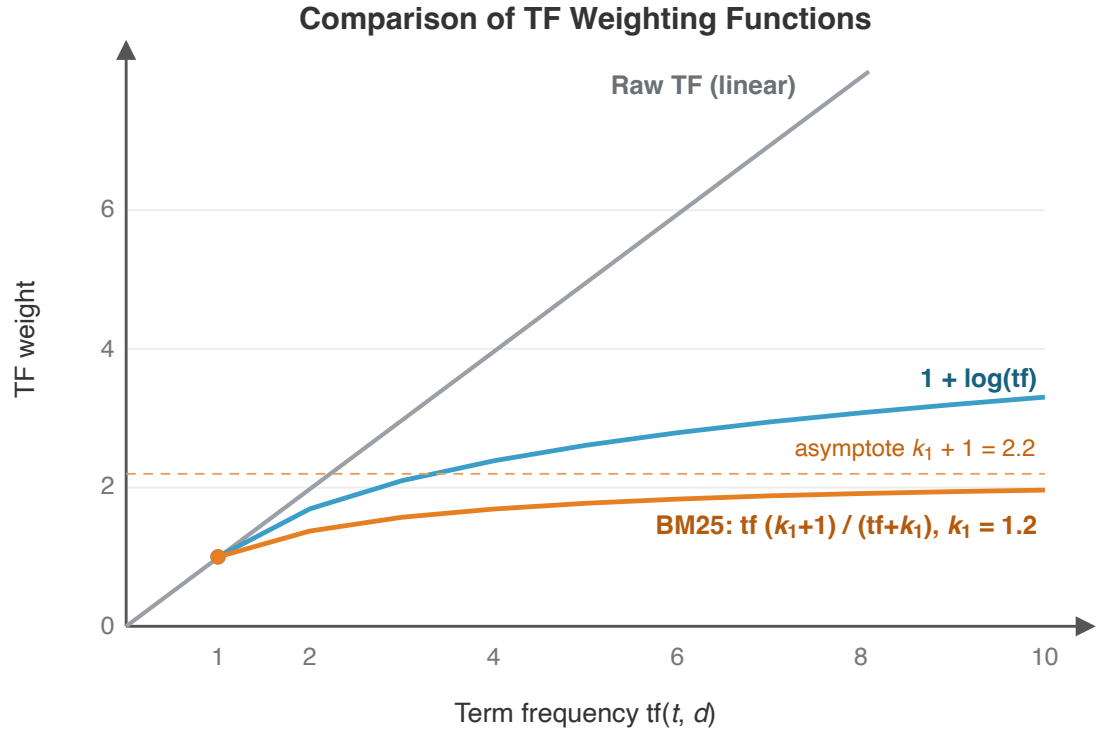
k_1: Parameter controlling the speed of TF saturation

- $k_1 = 0$: Completely ignores TF (binary model)
- $k_1 \to \infty$: Proportional to TF (no saturation)
- Typical value: $k_1 = 1.2$

b: Strength of document length normalization

- $b = 0$: No normalization
- $b = 1$: Full normalization
- Typical value: $b = 0.75$

l_d/L: Document length / Average document length



Saturation: even as tf grows, the weight gain diminishes
— 10 occurrences are not 10× as important as 1

BM25's saturation function is theoretically derived from the probabilistic model (2-Poisson)

Document length normalization: Longer documents naturally have higher term counts, so normalization by average document length ensures fairness

Let us concretely compute BM25 scores using the following example

Corpus settings:

Total documents: $N = 100{,}000$

Average document length: $L = 200$ words

Parameters: $k_1 = 1.2$, $b = 0.75$

Query: q = "Kyoto tourism"

Corpus statistics:

$\mathrm{df}(\mathrm{Kyoto}) = 5{,}000$

$\mathrm{df}(\mathrm{tourism}) = 10{,}000$

Document 1: Kyoto guidebook

Document length: $l_{d_1} = 300$ words

$\mathrm{tf}(\mathrm{Kyoto}, d_1) = 15$

$\mathrm{tf}(\mathrm{tourism}, d_1) = 8$

Document 2: Tokyo tourism guide

Document length: $l_{d_2} = 150$ words

$\mathrm{tf}(\mathrm{Kyoto}, d_2) = 0$

$\mathrm{tf}(\mathrm{tourism}, d_2) = 12$

Question: Which document gets a higher score?

Step 1: Compute the IDF for each query term

$$\mathrm{IDF}(\mathrm{Kyoto}) = \log_{10} \frac{N - \mathrm{df} + 0.5}{\mathrm{df} + 0.5} = \log_{10} \frac{100{,}000 - 5{,}000 + 0.5}{5{,}000 + 0.5} = \log_{10} \frac{95{,}000.5}{5{,}000.5} \approx \log_{10}(19.0) \approx 1.28$$

$$\mathrm{IDF}(\mathrm{tourism}) = \log_{10} \frac{100{,}000 - 10{,}000 + 0.5}{10{,}000 + 0.5} = \log_{10} \frac{90{,}000.5}{10{,}000.5} \approx \log_{10}(9.0) \approx 0.95$$

"Kyoto" (IDF = 1.28) has a **higher IDF** than "tourism" (IDF = 0.95)

"Kyoto" is relatively rare in the corpus, so it has greater discriminative power for search

$\log$ is the common logarithm (base 10), consistent with the exercises

Step 2: Compute the document length normalization coefficient B, the TF component, and multiply by IDF

Document 1 ($l_{d_1} = 300$, longer than average):

$$B_1 = 1 - b + b \cdot \frac{l_{d_1}}{L} = 1 - 0.75 + 0.75 \times \frac{300}{200} = 0.25 + 1.125 = 1.375$$

Term	IDF	TF component = $\frac{\text{tf} \times 2.2}{\text{tf} + 1.2 \times 1.375}$	Contribution
Kyoto	1.28	$\frac{15 \times 2.2}{15 + 1.65} = \frac{33.0}{16.65} \approx 1.98$	**2.53**
tourism	0.95	$\frac{8 \times 2.2}{8 + 1.65} = \frac{17.6}{9.65} \approx 1.82$	**1.73**

$$\text{BM25}(q, d_1) = 2.53 + 1.73 = \mathbf{4.26}$$

Document 2 ($l_{d_2} = 150$, shorter than average): $B_2 = 0.25 + 0.75 \times 0.75 = 0.8125$

$$\text{BM25}(q, d_2) = 0 + 0.95 \times \frac{12 \times 2.2}{12 + 1.2 \times 0.8125} = 0.95 \times \frac{26.4}{12.975} \approx \mathbf{1.93}$$

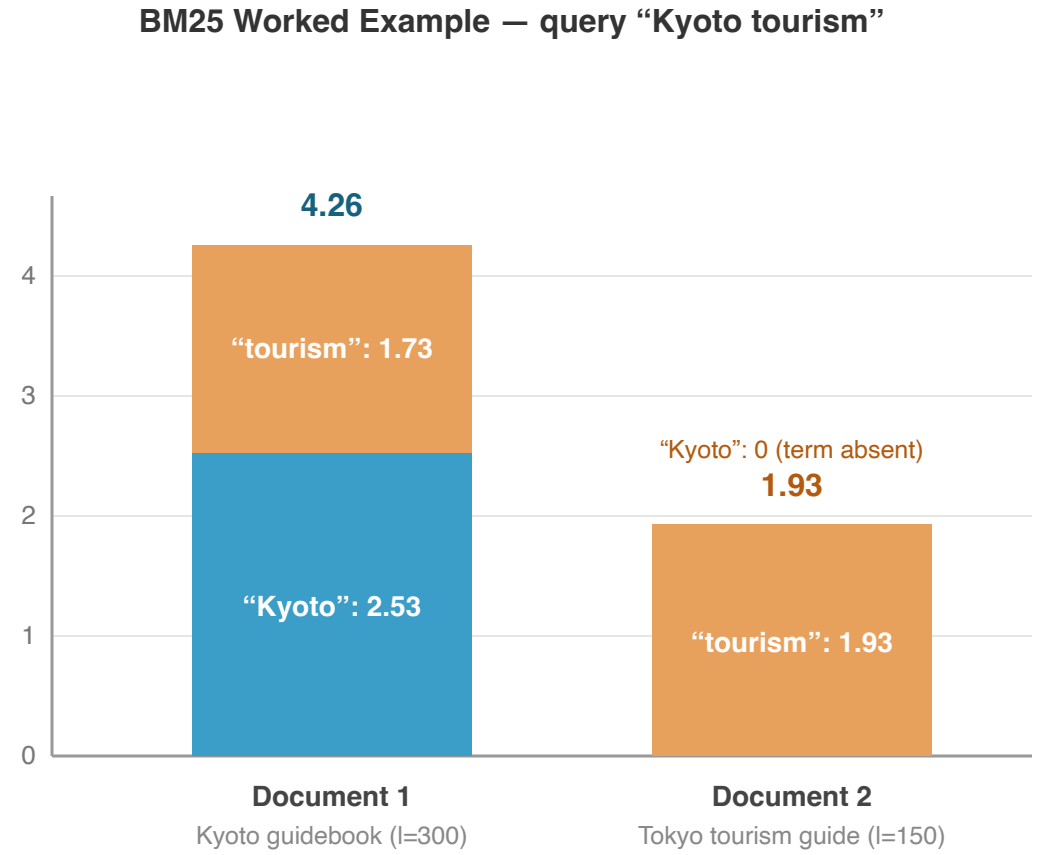
Document 1 (Kyoto guidebook) scores higher than Document 2 (Tokyo tourism guide)

	Document 1	Document 2
Content	Kyoto guidebook	Tokyo tourism guide
"Kyoto" contribution	**2.53**	0 (not present)
"tourism" contribution	1.73	1.93
Total	**4.26**	1.93

Interpretation:

The **high IDF** (= rarity) of "Kyoto" made the decisive difference

Document 2 contains "tourism" frequently, but scores low because it lacks the key query term "Kyoto"



BM25 reflects both **query term coverage** and **discriminative power of each term (IDF)**

Several variants of BM25 have been proposed to address practical issues

BM25+ [Lv & Zhai, 2011]

- BM25 has a problem where the TF component approaches zero for very long documents
- BM25+ adds a constant δ to guarantee that a term's occurrence **always contributes a positive score**
- Corrects the anomaly where "a term appears but gets the same score as if it did not"

BM25F [Robertson & Zaragoza, 2009]

- An extension for **structured documents** (title, body, anchor text, etc., with multiple fields)
- Each field has its own length normalization parameter and weight
- In Lucene/Elasticsearch, "field-weighted search," conceptually similar, is widely used in practice

BM25 Family Comparison

	BM25	BM25+	BM25F
Target	Single field	Single field	Multiple fields
TF lower bound for long docs	Approaches 0	$\delta > 0$	---
Field weights	None	None	Yes
Use case	General purpose	Long document collections	Web search

Name origin: BM = "Best Match". The 25th scoring function tried by Robertson et al. in the Okapi system

Even though it is called "BM25," there are non-negligible performance differences across implementations

Sources of differences:

1. **Many variants**: Numerous variants have been proposed since the original [Trotman et al., 2014]
2. **Different preprocessing**: Document cleaning, stop word lists, tokenizers, stemmers
3. **Parameter settings**: Many studies do not specify k_1 and b values

Concrete example (MS MARCO leaderboard):

Anserini's BM25: MRR@10 = 0.186

Microsoft's BM25 baseline: MRR@10 = 0.165

The same "BM25" with a **2-point difference**

Important Lesson

When claiming the effectiveness of a new ranking model, **scrutinize the quality of the baseline**

Beware of "illusion of improvement" from comparison with weak BM25 baselines

Armstrong et al. [2009] meta-analysis: Concluded that across major IR conference papers from 1998-2008, there was **no evidence of substantial improvement**

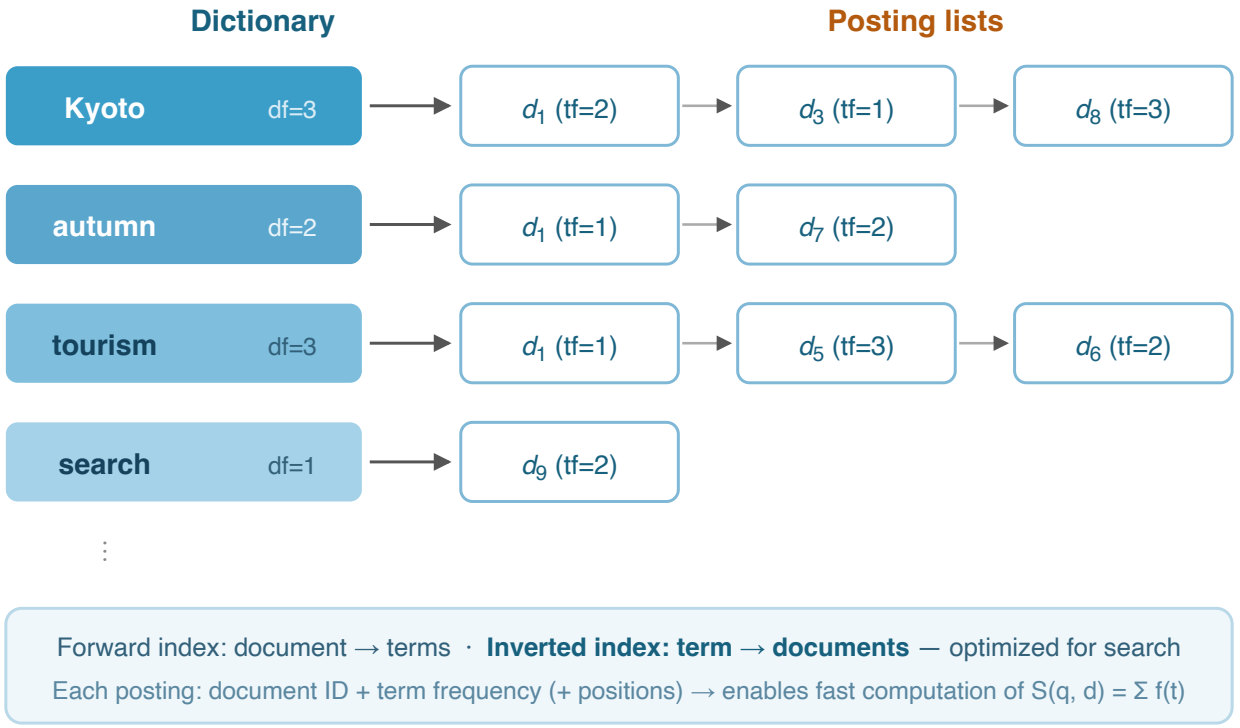
Trotman et al., "Improvements to BM25 and Language Models Examined", ADCS, 2014
Armstrong et al., "Improvements That Don't Add Up", CIKM, 2009

Inverted Index

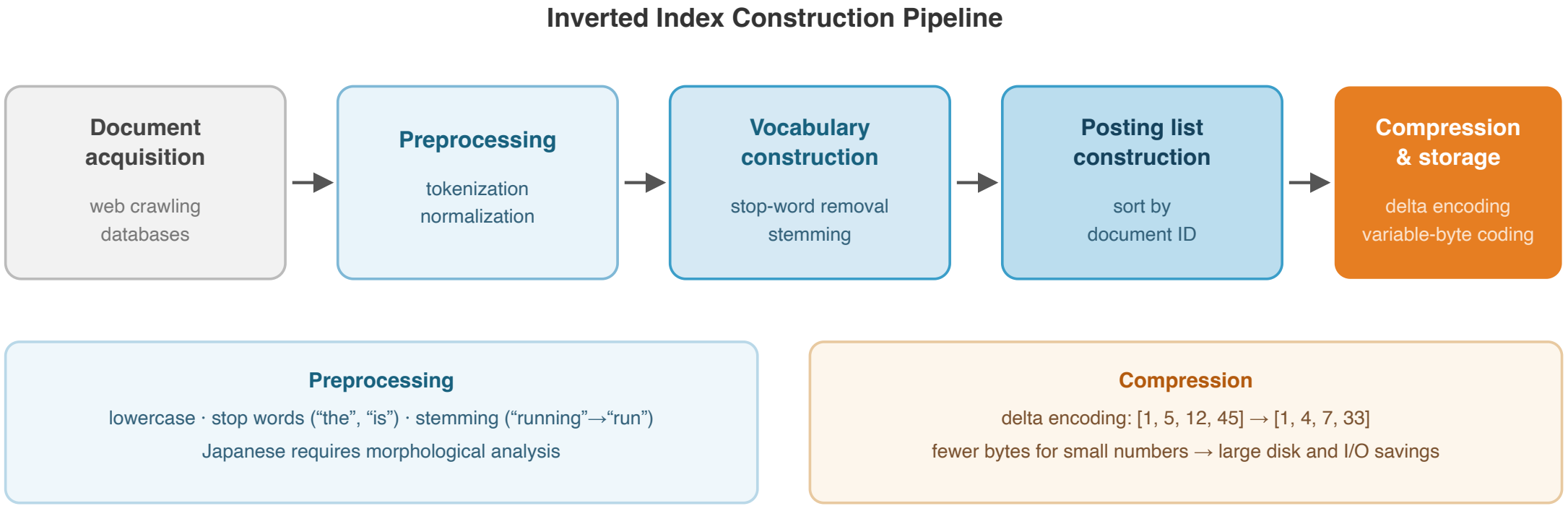
Keyword search is almost always implemented using an **Inverted Index**

> **Inverted Index**: A data structure that maintains, for each term, a list of documents containing that term (the **posting list**)

- Forward index: document -> terms (which terms does this document contain?)
- **Inverted** index: term -> documents (which documents contain this term?)
- Each posting list entry: **document ID + term frequency** (+ position information)
- Enables efficient computation of scoring functions $S(q,d) = \sum_{t \in q \cap d} f(t)$



The pipeline for building an inverted index from a document collection



Posting lists can be huge — compression is essential at web scale

The **mainstream approach** in modern search engines

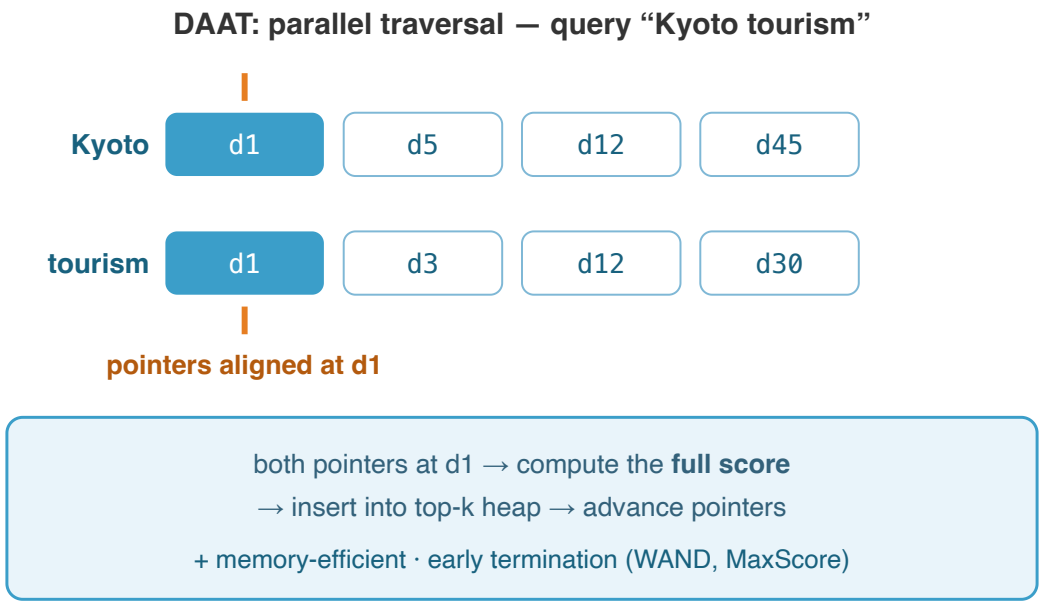
How it works:

1. Open posting lists for all query terms simultaneously
2. Advance pointers in parallel, finding the next document that appears in any list
3. Compute the **complete score** for that document (summing contributions from all matching terms)
4. Insert into a top-k heap
5. Move to the next document

Advantages:

Memory efficient — only one document's score is in memory at a time

Can use **early termination** (WAND, MaxScore) to skip low-scoring documents



An older approach, less common in modern engines

How it works:

1. Process one query term at a time
2. Traverse its entire posting list
3. Add each document's partial score to an **accumulator** (a hash map: doc_id → partial_score)
4. After processing all query terms, sort accumulators and return top-k

Disadvantages:

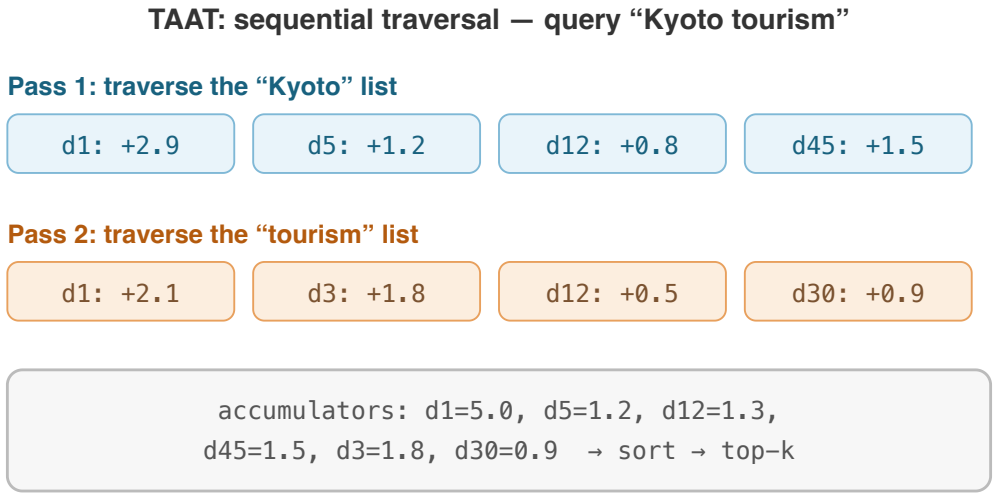
Requires accumulators for **all candidate documents** — high memory usage

Cannot easily apply early termination optimizations

Efficiency techniques (applicable to both):

Skip pointers: Fast forward jumps within posting lists

Dynamic pruning (WAND, MaxScore): Skip documents that cannot enter the top-k



– accumulators for all candidates (high memory)
· early termination is difficult

Both DAAT and TAAT can use skip pointers within posting lists

Query Expansion and RM3

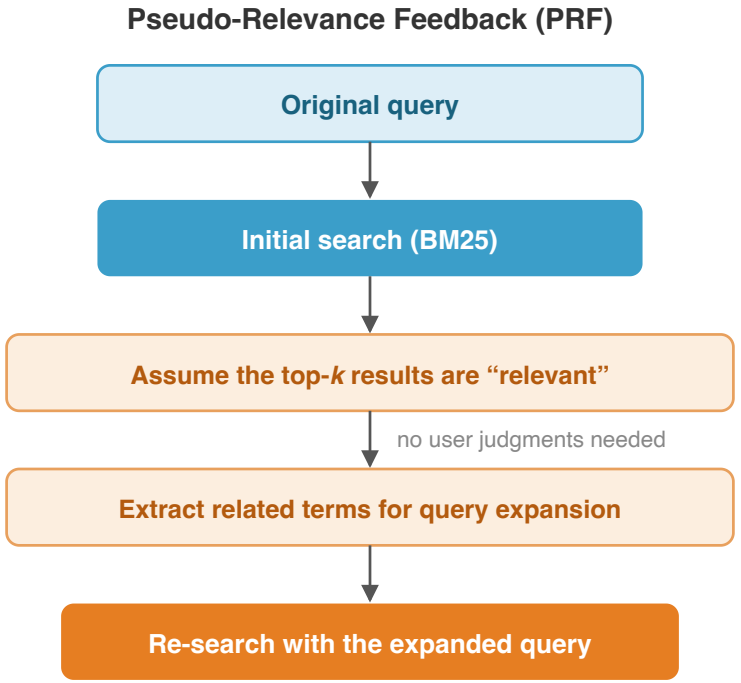
One of the most practical countermeasures against the vocabulary mismatch problem

Relevance Feedback [Rocchio, 1971]:

- Extract terms from documents that the user judges as "relevant" and add them to the query
- Highly effective, but the burden of requiring user judgments is a challenge

Pseudo-Relevance Feedback (PRF):

- An approach that eliminates the need for user judgments
- **Assumes the top-ranked documents from an initial search are relevant** and extracts terms from them
- Effectiveness has been demonstrated over decades
- Risk of being counterproductive when the assumption that "top results are good" fails



Origin: relevance feedback with explicit user judgments (Rocchio, 1971) — PRF removes the user burden by trusting the initial top results

Risk: counterproductive when the “top results are good” assumption fails (query drift)

RM3 (Relevance Model 3) is the most widely used pseudo-relevance feedback method

Original query: "Kyoto tourism" -> Initial search with BM25 -> Top 10 results used as pseudo-relevant documents

Step 1: Initial Search

Search "Kyoto tourism" with BM25 and retrieve top 10 results

→

Step 2: Extract Related Terms

Extract high-frequency terms from top documents
"temple" "Kiyomizu" "Arashiyama" "autumn leaves" "Gion"...

→

Step 3: Query Expansion

Interpolate original query with extracted terms
$q' = \alpha \cdot q + (1-\alpha) \cdot q_{RM1}$

→

Step 4: Re-search

Search again with expanded query using BM25

Effect: Even when searching for "Kyoto tourism," documents containing "temple," "Arashiyama," or "Kiyomizu" become retrievable -> Alleviates vocabulary mismatch

Abdul-Jaleel et al., "UMass at TREC 2004", TREC, 2004. "BM25 + RM3" is a strong keyword search baseline

To claim that a new method is "better than existing methods," the quality of the comparison target matters

Armstrong et al. [2009]

Meta-analyzed papers from major IR conferences, 1998-2008

Conclusion: "There is no evidence that ad hoc retrieval technology has improved over the past 10 years"

Cause: **"Illusion of improvement"** from comparison with weak baselines

Many papers claimed "improvement" compared to poorly tuned BM25

Yang et al. [2019]

Analyzed pre-BERT neural models

Could not outperform BM25 + RM3 baselines

Clear improvement was achieved only after the arrival of BERT

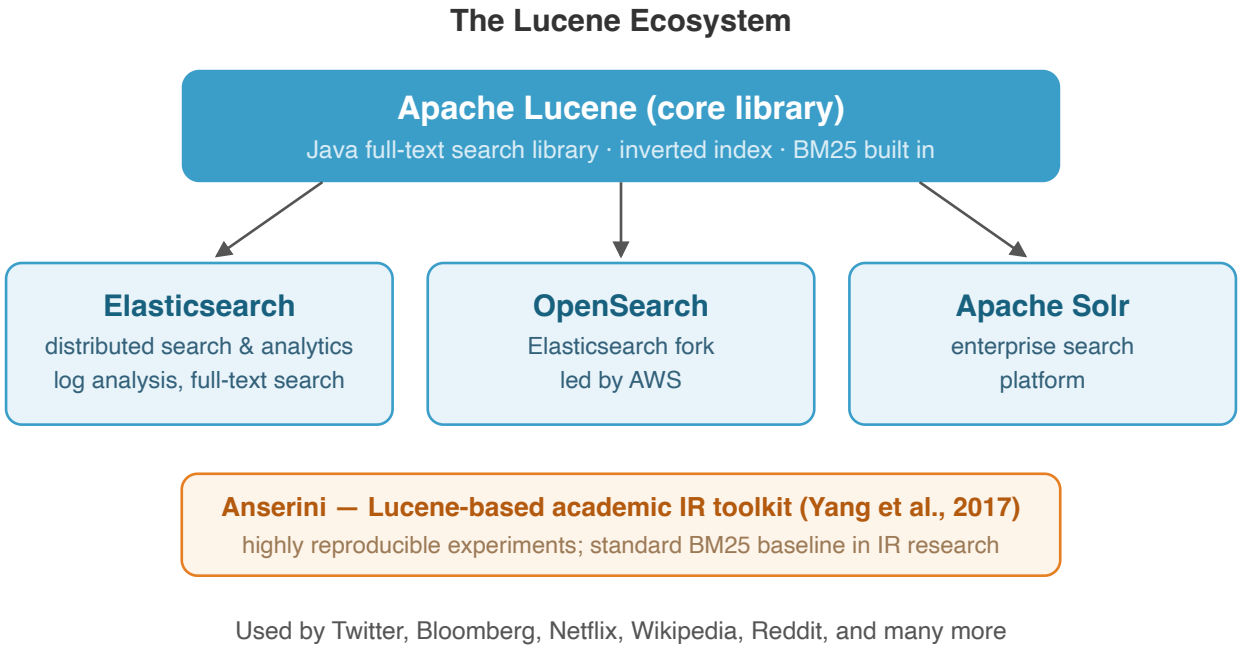
Lessons for Research

1. Baselines should be **properly tuned**
2. BM25 parameters (k_1, b) and preprocessing should be disclosed
3. When possible, include **BM25 + RM3** as a baseline
4. Use the same test collections and metrics for fair comparison with prior work

Search Engine Implementation Infrastructure

In practical search systems, **Apache Lucene** is the de facto standard

- **Apache Lucene**: A full-text search library written in Java
  - Builds and searches inverted indexes at high speed
  - Implements BM25 as the standard scoring function
- Search platforms built on Lucene:
  - **Elasticsearch**: Distributed search and analytics engine. Widely used for log analysis and full-text search
  - **OpenSearch**: Fork of Elasticsearch (led by AWS)
  - **Apache Solr**: Enterprise search platform
- Users include: Twitter, Bloomberg, Netflix, Wikipedia, Reddit, etc.



Searching Japanese text requires different preprocessing than English

Morphological Analysis Is Essential

Since Japanese has no spaces between words, **morphological analysis** is needed for tokenization

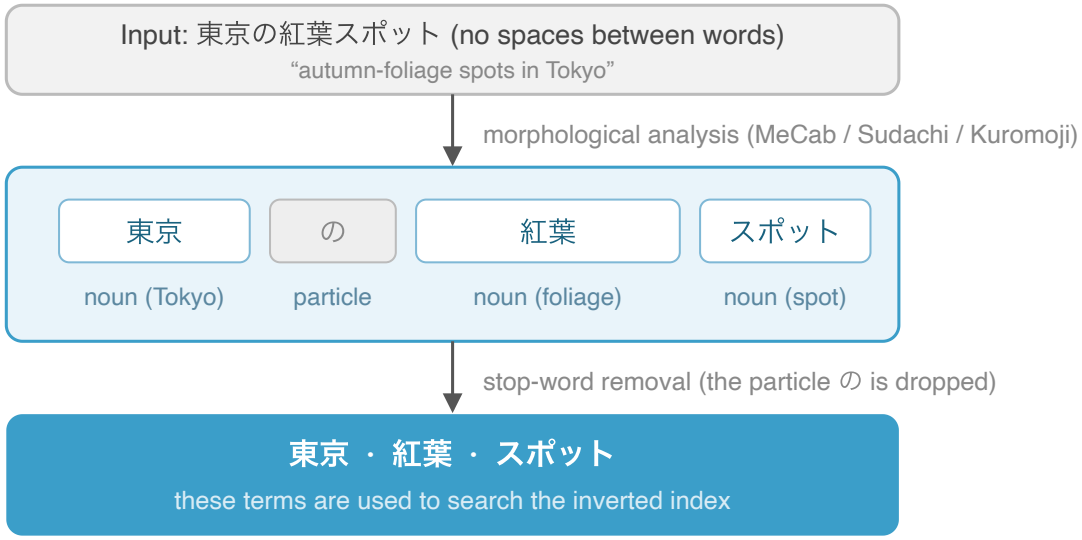
Major morphological analyzers:

MeCab: A fast morphological analyzer. Widely used in academic research

Sudachi: Developed by Works Applications. Supports multiple granularity levels

Kuromoji: A Japanese analyzer bundled with Lucene (typically used as a plugin in Elasticsearch)

N-gram Indexing: Character N-grams (bi-grams) supplement morphological analysis, handling unknown words and neologisms



N-gram indexing (character bi-grams) supplements morphological analysis, handling unknown words and neologisms

Modern search systems combine **multiple stages** of ranking to balance accuracy and efficiency

Stage 1 (Candidate Generation): Keyword search (BM25) quickly narrows down candidates

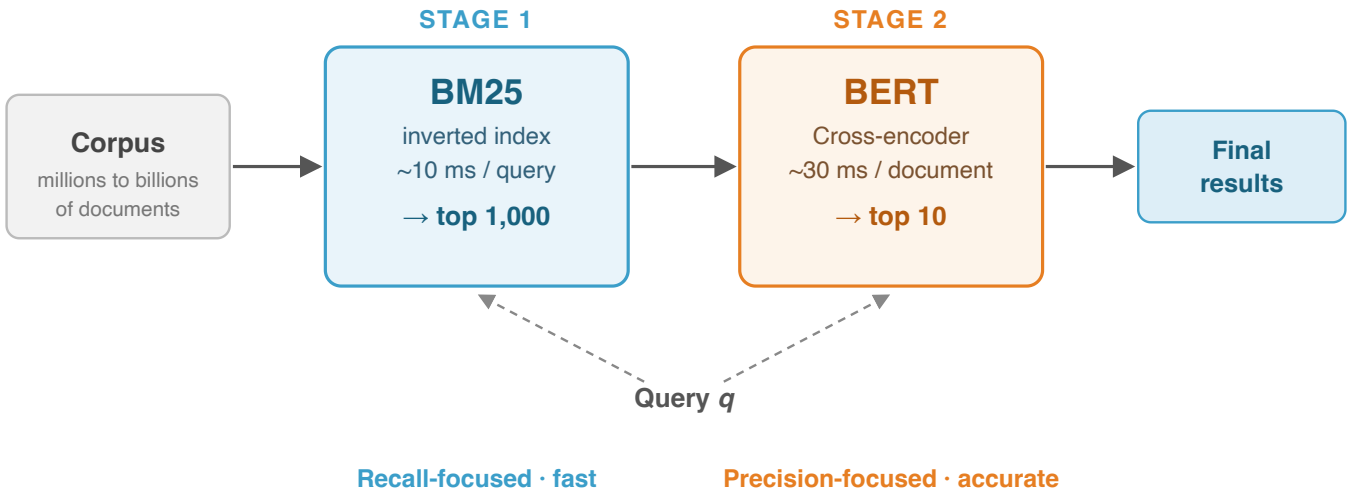
- Top k results in milliseconds via inverted indexes

Stage 2 (Reranking): A more accurate model reranks the candidates

- Applies expensive models (BERT, etc.) to few candidates

This configuration is called the **multi-stage ranking architecture**

In subsequent lectures, we will cover neural models for Stage 2



1. **Document ranking** is a foundational technology with broad applications beyond search, including QA, information filtering, and dialogue systems
2. The history of IR began with Bush's 1945 vision and evolved from manual -> automatic indexing -> TF-IDF -> BM25
3. **BM25** has a structure of IDF (term rarity) x TF saturation (diminishing frequency effect) + document length normalization, and is the de facto standard for keyword search
4. **Inverted indexes** enable fast retrieval from large-scale collections
5. The **vocabulary mismatch problem** is the fundamental limitation of exact match search, addressed by query expansion (RM3) and neural IR

To go beyond the limits of BM25:

- BM25 relies on exact term matching -> Weak against vocabulary mismatch
- Parameters (k_1, b) are manually set -> Room for optimization

Next lecture topics:

- **Learning to Rank**: Automatically learning ranking functions with machine learning
  - Pointwise / Pairwise / Listwise
  - LambdaMART
- **Neural ranking models**: DSSM, DRMM
- **The arrival of BERT**: Search revolution on MS MARCO

From keyword search (BM25) to machine learning-based ranking. Breaking through the limits of manual design with a data-driven approach